Chapter 3 | Processes

Kadir Has University | CCIP504 | Operating Systems & System Programming

Name: _____ Student ID: _____

Format: Work individually, then compare explanations with a partner. Core activities: 85 minutes. Optional coding extension: 20 minutes. Write reasons, not only keywords.

Tools: Linux or WSL with Bash and GCC for the practical tasks. Without a terminal, complete the prediction questions and mark observations as not run. No administrator access is required. Save code in your own working directory.

1 | From a program to a process - 10 min

Key terms. A program is stored code and data. A process is an executing instance, with an identity, address space and tracked execution state. A thread is an execution path within a process.

1.1 You launch the same calculator executable twice. How many process instances are normally created? Name two things that may differ between them.

1.2 Match each item to its role. Use: instructions; function-call state; dynamically allocated storage; saved execution position; kernel bookkeeping.

Item	Role
Text section	
Stack	
Heap / allocation mappings	
Program counter	
PCB (process control block)	

1.3 A process is ready but has not run for 5 ms. Does it still exist? Is its entire memory stored inside its PCB?

1.4 Can one process contain multiple threads? On one CPU core with one hardware execution context, can two threads execute instructions at exactly the same instant?

2 | Follow the process states

10 min | Slides 7-8, 12-15 | Distinguish runnable work from blocked work

Key terms. Ready: able to run once assigned CPU time. Running: executing instructions. Waiting/blocked: needs an event before it can run. Use the simplified five-state model, not exact Linux state letters.

2.1 Complete the state after each event. The process begins in the new state. Assume one thread and one CPU execution context.

Event	State immediately after event
A. The OS admits the process to the runnable set.	
B. The dispatcher assigns it the CPU.	
C. It requests unavailable input and blocks.	
D. Input arrives; it has not received CPU time yet.	
E. It is selected and resumes execution.	
F. Its time slice expires; it can still run.	
G. It runs again, finishes and exits.	

2.2 Why does input completion normally move the task to ready rather than guarantee immediate execution?

2.3 A task repeatedly tests a shared flag in a loop. Is this busy waiting or blocking? Can it consume CPU time while the flag remains false?

Observe a real process

```
sleep 60 &
pid=$!
ps -o pid,ppid,stat,comm -p "$pid"
wait "$pid"
```

Run in one Bash session. The last command may wait up to 60 seconds. Use a second terminal for other work if needed. STAT can include extra flags; Linux R includes both running and runnable tasks.

2.4 Record the PID, PPID and STAT you observed. Why is one ps snapshot insufficient to reconstruct the complete process history?

3 | Scheduling and context

10 min | Slides 9-19 | Track the work needed to pause and resume

Key terms. The scheduler chooses runnable work. Dispatch transfers execution to it. A context switch saves one task's execution context and restores another's. A mode change changes privilege level.

3.1 Task A stops halfway through a calculation and B runs. Name three kinds of information needed so A can later continue correctly.

3.2 For each event, say whether a task switch is required, possible, or not implied. Explain briefly. A mode change alone does not settle the answer.

Event	Task switch? Why?
A calls a system call; the kernel returns to A immediately.	
A blocks on input; runnable B is then scheduled.	
A hardware interrupt arrives while A runs.	

3.3 A context switch takes 4 microseconds in a hypothetical measurement. Each cycle contains 996 microseconds of application work plus one switch. Calculate the switch overhead percentage of the 1000-microsecond cycle.

3.4 Compare a CPU-bound numerical calculation and an I/O-bound interactive tool. What is a CPU burst? Does running in the background automatically make a program CPU-bound?

3.5 Is saving CPU registers the same as swapping the entire process to disk? Explain what each operation is intended to achieve.

Think carefully: A textbook queue is a conceptual model. Real multicore systems can use per-CPU scheduling structures. Do not infer a universal FIFO scheduling policy from a queue drawing.

4 | Trace fork and private memory

15 min | Slides 20-24 | Predict first, then compile and run

Key terms. fork creates a child process. It returns 0 in the child, the child PID in the parent, or -1 on failure. Ordinary writable variables are logically separate after fork; inherited shared mappings are a different case.

Save as **fork_demo.c**. This single-threaded example checks process creation and retries waitpid when interrupted by a signal.

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    int x = 5;
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        x += 2;
        printf("child: x=%d, pid=%ld, ppid=%ld\n",
               x, (long)getpid(), (long)getppid());
        return 0;
    }
    pid_t r;
    do { r = waitpid(pid, NULL, 0); }
    while (r == -1 && errno == EINTR);
    if (r == -1) { perror("waitpid"); return 1; }
    printf("parent: x=%d, child=%ld\n", x, (long)pid);
    return 0;
}
```

```
gcc -std=c11 -Wall -Wextra fork_demo.c -o fork_demo
./fork_demo
```

4.1 Predict x in each output line. Which line comes first in a normal successful run, and what enforces that order?

4.2 Why does the child update not change the parent's x? Does copy-on-write mean their ordinary variables are shared application state?

4.3 Remove only the waitpid block in a copy of the program. What output-order guarantee is lost? Record one observed run, but do not treat that order as universal.

5 | Replace, wait and collect status

10 min | Slides 21-25 | Successful exec keeps the same process identity

Key terms. exec replaces a process image; it does not create another process. A zombie has exited but has not yet been reaped. An orphan has lost its parent while still alive.

Save as **exec_demo.c**. Here waitpid collects the status of one specific child.

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        execlp("echo", "echo", "Hello from exec", (char *)NULL);
        perror("execlp");
        _exit(127);
    }
    int status;
    pid_t r;
    do { r = waitpid(pid, &status, 0); }
    while (r == -1 && errno == EINTR);
    if (r == -1) { perror("waitpid"); return 1; }
    if (WIFEXITED(status))
        printf("exit status=%d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("signal=%d\n", WTERMSIG(status));
    return 0;
}
```

```
gcc -std=c11 -Wall -Wextra exec_demo.c -o exec_demo
./exec_demo
```

5.1 If execlp succeeds, does the child execute perror? Does its PID change? Predict the two normal output lines.

5.2 Replace the first "echo" argument with "ccip504_missing_command". What executes next, and which exit status should the parent report? Why check WIFEXITED first?

5.3 Classify: (a) child exited, parent has not collected status; (b) parent exited while child still runs. Must situation (b) cause situation (a)?

6 | Producer-consumer ring buffer

10 min | Slides 29-32 | Paper trace; conceptual pseudocode, not runnable C

Model. BUFFER_SIZE = 5. Index in points to the next insertion; out to the next removal. Empty: in == out. Full: (in + 1) % 5 == out. One slot is deliberately reserved.

```
Producer: wait until not full; buffer[in] = item;
          in = (in + 1) % 5;
Consumer: wait until not empty; item = buffer[out];
          out = (out + 1) % 5; consume(item);
```

6.1 Start with in = 0, out = 0 and no items. Apply each operation once, in order. If it cannot proceed immediately, mark WAIT and leave the state unchanged; do not carry that attempted operation into the next row.

Operation	in	out	Items stored	Done / WAIT
produce A				
produce B				
consume				
produce C				
produce D				
produce E				
produce F				

6.2 What is the usable capacity? Why would using all five slots make in == out ambiguous in this representation?

6.3 Trace a removal when out = 4. What index follows, and why is modulo used?

6.4 Replace "wait until" with while (condition) { }. Does the caller enter a blocked state automatically? Name a mechanism that could support a blocking design.

6.5 Is this pseudocode enough for safe concurrent C? Explain the need for synchronization and why an ordinary local array is not automatically shared between processes.

7 | Choose an IPC design

10 min | Slides 26-28, 33-49 | Explain both data transfer and coordination

Key terms. IPC means interprocess communication. Shared memory exposes a shared region; message passing transfers information through communication operations. A socket is an endpoint. RPC packages a remote operation as a call and reply.

7.1 Choose a plausible mechanism and justify it. Several designs may be defensible; state your assumptions.

Scenario	Mechanism + reason
Two local processes exchange large image frames repeatedly.	
Several workers take jobs from one shared work queue.	
A client requests a calculation from another machine.	

7.2 A bounded queue holds two messages and is full. A blocking sender attempts a third send. What can happen? How does a nonblocking send differ? Does successful enqueue mean a worker has already processed the message?

7.3 Two workers can receive from mailbox A. One message is sent. Must both workers receive a copy? What extra semantics would broadcasting require?

7.4 An RPC times out after a server may have performed a payment. Does timeout prove the payment failed? Why could blindly retrying be harmful?

7.5 Match each shared-memory family to its typical calls. Write System V or POSIX shared memory.

Calls	Family
shmget, shmat, shmdt, shmctl	
shm_open, ftruncate, mmap, munmap, shm_unlink	

7.6 For TCP, does one send guarantee exactly one corresponding receive of the same length? Name one way an application can mark message boundaries.

8 | Connect processes with pipes

10 min core + 20 min optional extension | Slides 50-54

Key terms. A Unix pipe is a unidirectional byte stream. pipe(fd) sets fd[0] to the read descriptor and fd[1] to the write descriptor; those positions are not necessarily descriptor numbers 0 and 1.

Run a shell pipeline

```
printf 'alpha\nbeta\ngamma\n' | wc -l
```

8.1 Record the output. Which command produces bytes and which consumes them? Does the pipe write those bytes into an ordinary disk file?

8.2 A parent creates a pipe, then forks. Both processes initially inherit both ends. Complete a parent-to-child design.

Process	Close unused end	Use which end?
Parent		
Child		

8.3 The parent closes its write end after sending. The child reads all bytes but still holds its own write-end reference open. Why might its next blocking read wait instead of returning EOF?

8.4 Can two sibling child processes communicate using a pipe created by their common parent? Explain how they obtain the endpoints.

Optional coding extension - 20 min

Build pipe_demo.c: create one pipe before fork; parent closes the read end, sends "Hello child\n" and closes the write end; child closes its write end, reads until EOF and prints the received bytes; parent reaps the child. Check errors. Retry read/write/waitpid on EINTR; loop over partial reads/writes. Compile with -std=c11 -Wall -Wextra. Use man 2 pipe, man 2 read and man 2 write for help.

Extension evidence: record output and explain which close allows the child to see EOF.

Exit ticket

In three short statements, distinguish ready/waiting, fork/exec, and shared memory/message passing.

Reading: Operating System Concepts, 10th ed., Chapter 3; annotated course slides, Chapter 3 (historical examples are qualified in their speaker notes). Linux references: man7.org/linux/man-pages/ - fork(2), wait(2), exec(3), pipe(7). Checked 3 November 2026.